

≡ Hide menu

CUDA Kernels

Converting CPU Divide and Conquer to Parallel Kernel Execution

Defining Kernel Thread and Data Layout via Blocks

3-Dimensional Thread and Data Layout using Grids and Blocks

Execution of CUDA Kernel in 3-Dimensional Data/Thread Space

Video: Multidimensional Gaussian Blur Kernel Example

7 min

Practice Quiz: Multidimensional Data and Computation on the GPU Quiz

Started

Programming Assignment: Performing I/O to Grayscale on Image Data Assignment

2h

Video: Module2 Image Processing Assignment Overview Videos

6 min

🎉 Congratulations! You passed!

Grade received 100% To pass 70% or higher

Multidimensional Data and Computation on the GPU Quiz

Go to next item

1. Which statement is not true about the syntax below: 1 / 1 point

kernel<<<a,b>>>(args)

Review Learning Objectives

☐ The variable b describes the layout of threads within blocks.

☐ The variable a can be a dim3 type or an integer

☒ The value(s) in args, can only be the name(s) of a kernel(s).

☐ The kernel that is called by the kernel launch function is not a particular thread division layouts.

Help me practice

Let's chat

☒ Correct

This is false as the value(s) that args represents are arguments to the kernel.

Submit your assignment

Try again
2. If you would like a 2-dimensional data/execution mapping of threads in your blocks for your kernel, which of the following would calculate the index of the thread and data? 1 / 1 point

Note: that N is defined as 512

70% or higher

100% Pass

We keep your highest score

☒ int x = blockIdx.x * blockDim.x + threadIdx.x;
int y = blockIdx.y + blockDim.y + threadIdx.y;
int index = x + y * N;

Like

Dislike

Report an issue

☐ int a = blockIdx.y * blockDim.x + threadIdx.y;
int b = blockIdx.y + blockDim.y + threadIdx.y;
int index = a + b * N;

☐ int x = blockIdx.y * blockDim.x + threadIdx.y;
int y = blockIdx.y + blockDim.z + threadIdx.y;
int index = x + y * N;

☐ int a = blockIdx.y * blockDim.x + threadIdx.y;
int b = blockIdx.y + blockDim.y + threadIdx.y;
int index = x + y * N;

☒ Correct

This is the correct answer.
3. If you would like a 3-dimensional data/execution mapping of threads in your blocks for your kernel, which of the following would calculate the index of the thread and data? 1 / 1 point

Note: that N is defined as 512

☐ int index = N * N * N

☐ int index = blockIdx.x + blockIdx.y + blockIdx.z;

☒ int index = blockIdx.x * blockDim.x * blockDim.y + blockDim.y + blockDim.z * threadIdx.z * blockDim.y * blockDim.x + threadIdx.y * blockDim.x + threadIdx.x;

☐ float index = blockIdx.x * blockDim.x * blockDim.y + blockDim.z * threadIdx.z * blockDim.y * blockDim.x + threadIdx.y * blockDim.x + threadIdx.x;

☒ Correct

This is correct because the index variable calculates the x, y, and z offsets based on the threadIdx variables for x,y,z values and the height, width, and depth of the 3D thread-block space.
4. Given the following code how many threads will be executed in a single block (the variable threadsPerBlockPerDimension)? Note: math.pow(x,2) returns x squared. 1 / 1 point

#include <math.h>
#define N 16384
int blocksPerGridPerDimension = 2;
dim3 gridLayout(blocksPerGridPerDimension, blocksPerGridPerDimension);
int threadsPerBlockPerDimension=N/math.pow(blocksPerGridPerDimension,2);

☐ 8192

☐ 2048

☐ 1028

☒ 4096

☒ Correct

This is the correct value, since 16384/(2^2)=4096.
5. Given the following code, how would you change the initialization of the dim3 block-variable to create blocks of 256 threads? You are able to change the dimensions of the dim3 for the block and the associated code to retrieve the thread index. 1 / 1 point

dim3 grid(32,32);
dim3 block(1,1);
kernel<<<grid,block>>>(args);

☐ dim3 block(2,2,2);

☐ dim3 block(4,4, 4,4);

☒ dim3 block(8,4,8)

☒ Correct

This is correct as 8*4*8 = 256

☐ dim8 block(16,16)

☐ dim3(16,16)
6. Given the following kernel code (executable from host code), choose a **bolded** area to correct and the correct code snippet from the options below: 1 / 1 point

__device__ matrixAdd(int *matrix_a, int *matrix_b, int *matrix_c){
int blockId = blockIdx.x + blockIdx.y * gridDim.x + gridDim.x * gridDim.y * blockIdx.z;
int threadId = blockId * (blockDim.x * blockDim.y * blockDim.z)
+ (threadIdx.z * (blockDim.x * blockDim.y))
+ (threadIdx.y * blockDim.x) + threadIdx.x;
c[threadId] = a[blockId] + b[threadId];
}

☒ __global__ matrixAdd(int *matrix_a, int *matrix_b, int *matrix_c){

☒ Correct

This is a correct answer, as the __device__ keyword is only for functions that are called from GPU devices, not the CPU host.

☐ int threadId = blockId.x * (blockDim.x * blockDim.y * blockDim.z)

☒ + (threadIdx.y * blockDim.x) + threadIdx.x;

☒ Correct

This is a correct answer because the original statement had the y thread offset calculated by using the z dimension of the block dimension, which can give the wrong dimensional offset when the dimension of y and z are not equal in the size in threads.

☐ c[threadId] = a[blockId] - b[threadId];
- https://www.coursera.org/learn/introduction-to-parallel-programming-with-cuda/quiz/PCU5o/multidimensional-data-and-computation-on-the-gpu-quiz/attempt?directToCourse=true
- 1/1